{PROXY}Maze'26

Proxy Technical Reference

From zero to running proxy requests in your code.

Torch Labs Software | Challenge 2 Supporting Material

1. What Is a Proxy and How Does It Work

When your code makes a request, it travels across the internet with your IP address attached. The server you're calling sees that IP and decides whether to respond, rate-limit, or block.

A proxy sits between your code and the target. Your request goes to the proxy server first. The proxy forwards it to the target using its own IP. The target sees the proxy's IP, not yours.

> Your code sends to the proxy server. The proxy forwards to the target. The response comes back the same way. The target never sees your real IP or your server's datacenter block.

Why this matters for your build

A standard HTTP request from your server comes from a datacenter IP. Websites know datacenter IP ranges and block them automatically. This isn't a bug in your code. It's an intentional anti-bot measure on the site's side.

Residential proxies route your requests through real home internet connections. From the website's perspective, the request looks like it came from an ordinary person at home. That's why residential proxies get through where datacenter requests don't.

2. Why Websites Block Automated Requests

Understanding the blocking mechanisms helps you build around them. Websites use several layers:

IP reputation

Websites check whether the requesting IP belongs to a known datacenter, VPN, or proxy provider. Datacenter IPs have low trust scores. Real residential IPs from ISPs have high trust scores. This is the primary reason proxies are needed.

Request rate

500 requests in 10 seconds from one IP is not human behavior. Rotating residential proxies spread requests across many IPs so no single one triggers a rate limit.

Session consistency

If your IP changes mid-session during a login or checkout flow, the site detects that a human couldn't physically be in multiple locations at once. Sticky sessions solve this: you hold the same IP for the full session.

> Your proxy handles the IP layer. If you're getting blocked, the first question is: are you using the right product for this use case? The second question is: are you sending too many requests too fast from a single session?

3. TorchProxies Products

You'll receive access to specific TorchProxies products based on what you stated in your proposal. Here's what each product is and when to use it.

Product	What it is	Use it when
Standard Residential	30M+ IPs in 195+ countries. Standard residential IPs suitable for general scraping, data collection, and automation. Rotating and sticky sessions. Country, state, and city targeting.	General web scraping, data collection, price monitoring, automated workflows that don't hit heavily protected targets.
Premium Residential	90M+ IPs in 195+ countries. Premium-grade residential IPs with ultra-low latency. Suited for advanced scraping and targets with stronger anti-bot measures.	Advanced scraping, targets with CAPTCHA layers, tasks where speed and IP quality both matter.
X Residential	120M+ IPs. Hybrid pool combining premium residential IPs and dedicated ISP pools. SOCKS5 support. Suitable for high-demand tasks and event sites.	High-detection targets, sneaker and event sites, use cases needing the highest trust score.
ISP Proxies	Static IPs registered under real ISPs. Fixed IP per proxy, no rotation. Unlimited bandwidth. Available in USA, UK, Germany, Australia, France, Netherlands, Italy, Hong Kong, Canada, Korea, Japan.	Login flows, session-based workflows, checkout automation, repeated portal access where a changing IP causes problems.

Which product to use for your build

- Scraping data at scale from many pages or sources: Standard or Premium Residential with rotation.
- Scraping targets with strong anti-bot measures (Cloudflare, Akamai, DataDome): Premium or X Residential.
- Login flows, portal automation, checkout sequences: ISP proxies or sticky residential sessions.
- Your product does both (collect data broadly and also authenticate with one portal): use rotating residential for collection and ISP or sticky residential for the authenticated part.

> Your proposal stated which product category you need. If your actual build turns out to need a different tier, contact Torch Labs before you start integrating. Switching later costs time.

4. Rotating vs Sticky Sessions

Residential proxies can work in two modes. The mode is controlled entirely by your proxy string. No dashboard change needed.

	Rotating	Sticky
IP per request	New IP on every connection	Same IP held for the session lifetime
How to set	No session parameter in the string	Add _session-YOURID_lifetime-Xm to the password
Use when	Scraping many pages, collecting data at scale	Logging in, maintaining a cart, posting data to a portal
Risk if wrong	Throttled by request rate	IP changes mid-session, looks suspicious to the target site

How rotation works

If there's no session parameter in your proxy string, the gateway assigns a fresh IP from the pool on every new connection. Nothing special to configure. Just omit the session and you get rotation.

How sticky sessions work

Add a session ID and a lifetime to the proxy string. The gateway keeps you on the same IP for the duration of the lifetime. Change the session ID and you get a new IP. Keep it and you stay on the same one.

Sticky sessions matter when the target site would flag an IP change mid-flow, for example logging in from one IP and then browsing the account from a different one.

5. Reading Your Proxy String

TorchProxies uses a standard authenticated format. Once you can read one proxy string, you can read all of them.

Residential proxy: rotating

The standard format is:

```
host:port:username:password-country-xx
```

A real example looks like:

```
proxy.torchlabs.xyz:9000:YOUR_USERNAME:YOUR_PASSWORD-country-us
```

Part	Example value	What it means
host	proxy.torchlabs.xyz	The gateway server. Your code connects here, not to the target site.
port	9000	Determines product tier and protocol. Standard HTTPS = 9000. See the ports table for your product.
username	YOUR_USERNAME	Your account identifier. Provided in the dashboard.
password	YOUR_PASSWORD-country-us	Your password plus any parameters. Treat the entire string after the last colon as one value.
country-us	appended to password	Geo-targeting. Exit from a US residential IP. Change to country-gb for UK, country-sg for Singapore, etc.

> No session parameter = rotating. You get a new IP every request. This is the default for data collection.

Residential proxy: sticky session

To hold the same IP across multiple requests, add session and lifetime parameters to the password:

```
proxy.torchlabs.xyz:9000:YOUR_USERNAME:YOUR_PASSWORD-country-us_session-myapp001_lifetime-30m
```

Parameter	What it does
`_session-myapp001`	Your session identifier. Keep this the same to hold the same IP. Change it to get a new one. You choose the value.
`_lifetime-30m`	How long the gateway keeps you on the same IP before it cycles. Options: 10m, 30m, 1h. Use 30m for most login flows.

ISP proxy

ISP proxies use the IP address directly. The IP you see in the string is the exact IP the target site will see:

```
IP_ADDRESS:PORT:YOUR_USERNAME:YOUR_PASSWORD
```

Example:

```
144.229.5.181:9931:YOUR_USERNAME:YOUR_PASSWORD
```

There are no session or country parameters. The IP is static. Username and password are just for authentication.

6. Port Numbers by Product

The port number determines both the product tier and the protocol. You don't write http:// or socks5:// in the string. The port handles it.

Product	HTTPS port	SOCKS5 port	Notes
Standard Residential	**9000**	**9003**	Default pool. Use for general scraping and data collection.
Standard Residential (EU)	**9001**	**9004**	EU exit nodes. Use when your target is European or you need EU geo-targeting.
Standard Residential (Asia)	**9002**	**9005**	Asia exit nodes.
Premium Residential	**8000**	**8003**	Premium pool. Lower latency, higher quality IPs.
Premium Residential (EU)	**8001**	**8004**	Premium EU exit nodes.
Premium Residential (Asia)	**8002**	**8005**	Premium Asia exit nodes.
X Residential (Global)	**6011**	**6014**	X pool global. Highest trust score.
X Residential (Global EU)	**6012**	**6015**	X pool EU exit nodes.
X Residential (USA)	**6000**	—	USA-specific X pool. HTTPS only.
X Residential (UK)	**6003**	—	UK-specific X pool. HTTPS only.

> Your dashboard will show which ports apply to your specific subscription. Use the port for your product tier. If you're unsure, start with the default HTTPS port and switch to SOCKS5 only if your tool requires it.

7. Geo-Targeting

Append country and city parameters to the password to control where your exit IP comes from.

Country targeting

```
# Exit from the United States
YOUR_PASSWORD-country-us

# Exit from the United Kingdom
YOUR_PASSWORD-country-gb

# Exit from Singapore
YOUR_PASSWORD-country-sg

# Exit from India
YOUR_PASSWORD-country-in
```

Country codes use the two-letter ISO 3166-1 format. US, GB, AU, DE, FR, IN, SG, JP, CA, LK work the same way.

City targeting

Add a city after the country for more specific targeting:

```
YOUR_PASSWORD-country-us_session-myapp_lifetime-30m

# Country + city
YOUR_PASSWORD-country-us-city-nyc_session-myapp_lifetime-30m
```

> City availability varies by country and pool. If no IP is available for the city you requested, the gateway falls back to country-level. Test geo-targeting before you build your whole pipeline around it.

8. Getting Your Credentials

TorchProxies will give each shortlisted team access credentials after the top 10 are announced on June 7. You'll receive these directly. No purchase needed.

What you'll receive

- Your proxy host and the port numbers for your product tier.
- Your username and base password.
- For ISP proxies: a list in IP:port:user:pass format.
- A link to the dashboard where you can generate proxy strings and check usage.

Checking your connection before you build

Once you have credentials, verify the proxy is routing before you write any product code:

```
import requests

HOST = 'proxy.torchlabs.xyz'
PORT = '9000'  # use your product's port
USERNAME = 'your_username'
PASSWORD = 'your_password-country-us'

proxy_url = f'http://{USERNAME}:{PASSWORD}@{HOST}:{PORT}'
proxies = {'http': proxy_url, 'https': proxy_url}

r = requests.get('https://ipinfo.io/json', proxies=proxies, timeout=30)
print(r.json())
```

If it's working, you'll see a JSON response with an IP, country, and org field. The org should show an ISP name, not a cloud provider. If you see your own IP, the proxy isn't routing.

9. Integrating Proxies in Your Code

Working examples for the most common setups. Replace the placeholder values with your actual credentials.

Python: Standard, Premium, or X Residential (rotating)

```
import requests

HOST = 'proxy.torchlabs.xyz'
PORT = '9000'          # Standard: 9000 | Premium: 8000 | X Residential: 6011
USERNAME = 'your_username'
PASSWORD = 'your_password-country-us'  # rotating, no session

proxy_url = f'http://{USERNAME}:{PASSWORD}@{HOST}:{PORT}'
proxies = {'http': proxy_url, 'https': proxy_url}

response = requests.get('https://example.com', proxies=proxies, timeout=30)
print(response.status_code)
```

Python: sticky residential session (same IP across requests)

```
import requests

HOST = 'proxy.torchlabs.xyz'
PORT = '9000'
USERNAME = 'your_username'
SESSION = 'myapp001'   # change this string to get a different IP
PASSWORD = f'your_password-country-us_session-{SESSION}_lifetime-30m'

proxy_url = f'http://{USERNAME}:{PASSWORD}@{HOST}:{PORT}'
proxies = {'http': proxy_url, 'https': proxy_url}

s = requests.Session()
s.proxies = proxies
r1 = s.get('https://example.com/login', timeout=30)
r2 = s.post('https://example.com/dashboard', data={'key': 'value'}, timeout=30)
# Both requests use the same IP because the session ID is identical
```

Python: ISP proxy (static IP)

```
import requests

# ISP format: ip:port:user:pass, no domain or session params
ISP_IP   = '144.229.5.181'
ISP_PORT = '9931'
ISP_USER = 'your_isp_username'
ISP_PASS = 'your_isp_password'

proxy_url = f'http://{ISP_USER}:{ISP_PASS}@{ISP_IP}:{ISP_PORT}'
proxies = {'http': proxy_url, 'https': proxy_url}

response = requests.get('https://example.com', proxies=proxies, timeout=30)
```

Python: scraping multiple pages with fresh IPs per request

```
import requests

HOST = 'proxy.torchlabs.xyz'
PORT = '9000'
USERNAME = 'your_username'
BASE_PASS = 'your_password'

def get_proxy(country='us'):
    password = f'{BASE_PASS}-country-{country}'
    url = f'http://{USERNAME}:{password}@{HOST}:{PORT}'
    return {'http': url, 'https': url}

urls = ['https://example.com/page/1', 'https://example.com/page/2']

for url in urls:
    proxy = get_proxy()
    try:
        r = requests.get(url, proxies=proxy, timeout=30)
        print(f'{url}: {r.status_code}')
    except Exception as e:
        print(f'{url}: failed: {e}')
```

Node.js: axios with residential proxy

```
const axios = require('axios');
const { HttpsProxyAgent } = require('https-proxy-agent');

const HOST = 'proxy.torchlabs.xyz';
const PORT = '9000';
const USERNAME = 'your_username';
const PASSWORD = 'your_password-country-us';

const proxyUrl = `http://${USERNAME}:${PASSWORD}@${HOST}:${PORT}`;
const agent = new HttpsProxyAgent(proxyUrl);

const response = await axios.get('https://example.com', {
  httpsAgent: agent,
  timeout: 30000,
});
console.log(response.status);
```

> For Node.js, install the proxy agent package first: npm install https-proxy-agent axios

10. Verifying Your Proxy Works

Always verify before building your pipeline around it.

IP check

```
import requests

proxies = {'http': 'YOUR_PROXY_URL', 'https': 'YOUR_PROXY_URL'}

r = requests.get('https://ipinfo.io/json', proxies=proxies)
data = r.json()
print('IP:     ', data.get('ip'))       # must differ from your own IP
print('Country:', data.get('country'))   # should match your country param
print('Org:    ', data.get('org'))        # should show an ISP, not a cloud
provider
```

Verify rotation

```
for i in range(3):
    r = requests.get('https://ipinfo.io/ip', proxies=proxies)
    print(f'Request {i+1}: {r.text.strip()}')

# Expected for rotating mode:
# Request 1: 92.240.111.203
# Request 2: 185.220.34.57
# Request 3: 94.177.45.12

# If all three show the same IP, you have a session param active unintentionally.
```

11. Common Errors

Error	Cause	Fix
407 Proxy Auth Required	Wrong username or password	Check credentials from your dashboard. Paste each field separately to catch typos.
Connection refused	Wrong host or port	Confirm the host and port for your product tier from the ports table above.
Connection timeout	Network blocking outbound port	Try the SOCKS5 port alternative. Check if your environment restricts outbound connections.
Same IP on every request	Sticky mode active unintentionally	Remove the _session-... and _lifetime-... parameters from your password.
403 from target site	Target blocked the exit IP	Change the country parameter or switch to a higher-tier product.
SSL errors	HTTPS tunnelling misconfigured	Use https:// as the proxy scheme, not http://. Set verify=False only for debugging.

> When asking Torch Labs for help, include the exact error message, which product you're using (Standard/Premium/X/ISP), and the relevant code snippet with credentials redacted. That gets you an answer faster than a general description.

Questions and Support

Contact the Torch Labs team through the official challenge channels. For proxy integration questions, include your product tier, the error you're seeing, and the relevant code. This cuts the back-and-forth.